

Capítulo 15

C++: um C melhor — Introdução à Tecnologia de Objeto

Exercícios de Autorrevisão — Resolução Didática

“Bem-vindo à segunda parte deste livro! Os Capítulos 1 a 14 lhe deram domínio sobre a linguagem C procedural; agora começamos a transição para C++ — a linguagem que estende C com classes, sobrecarga, referências, templates e tratamento de exceções. Este capítulo apresenta os recursos não-orientados-a-objeto de C++ (referências, sobrecarga de funções, argumentos default, templates, *inline*, resolução de escopo, *bool*) e oferece uma visão panorâmica da modelagem OOAD com UML, que será aprofundada a partir do Capítulo 16.”

1 Exercício 15.1 — Vocabulário do Capítulo 15

Preencha os espaços em cada uma das sentenças (a)–(f).

(a) **Várias funções com o mesmo nome operando sobre tipos ou números diferentes de argumentos**

Resposta: sobrecarga (de função).

A **sobrecarga de funções** (*function overloading*) é um dos recursos mais imediatamente úteis ao programador que migra de C para C++. Em C, cada função deve ter um nome *globalmente único*; assim, se quiséssemos somar dois inteiros e dois reais, teríamos que escrever `soma_int` e `soma_double`. Em C++, podemos chamar ambas simplesmente de `soma`: o compilador decide qual chamar examinando os *tipos* e a *quantidade* dos argumentos da chamada — o que se denomina *assinatura* da função.

Observação de Engenharia de Software

A sobrecarga é implementada pelo compilador através de um processo chamado *decoração de nomes* (*name mangling*) ou *deformação de nomes*: internamente, cada versão sobrecarregada recebe um nome único que codifica os tipos dos parâmetros. Por exemplo, `soma(int,int)` pode tornar-se `_Z4somaii` e `soma(double,double)` pode tornar-se `_Z4somadd`. O programador não precisa lidar com esses nomes; mas, ao examinar mensagens do *linker*, eles aparecem.

(b) **Habilita o acesso a uma variável global de mesmo nome que uma local**

Resposta: operador unário de resolução de escopo (`::`).

Se o programador declara, no escopo de uma função, uma variável local com nome idêntico ao de uma variável global, a variável global fica *ocultada* (*hidden*). Para acessá-la mesmo assim, basta prefixá-la com o operador `::`:

```
int x = 100;           // global
int main() {
    int x = 7;         // local, oculta a global
    cout << x          << endl; // imprime 7
    cout << ::x        << endl; // imprime 100
}
```

Boa Prática de Programação

Apesar de `::` resolver o conflito, a melhor prática é evitá-lo: dar nomes distintos a variáveis locais e globais elimina a ambiguidade na origem. O `::` unário torna-se valioso quando se acessa nomes globais de bibliotecas cujos identificadores não podemos renomear.

(c) Permite definir uma única função para muitos tipos de dados

Resposta: template (de função).

Os **templates de função** (*function templates*) levam a sobrecarga a um patamar superior: em vez de escrever uma versão de `max` para `int`, outra para `double` e outra para `char`, escrevemos uma só versão genérica:

```
template <typename T>
T max(T a, T b) { return (a > b) ? a : b; }
```

O compilador, ao encontrar `max(3, 4)`, gera automaticamente a *instanciação* apropriada (`max<int>`). Esse mecanismo é a base da **programação genérica** e da Standard Template Library (STL) que estudaremos no Capítulo 21.

(d) Esquema de representação gráfica mais usado para modelagem OO

Resposta: a UML (*Unified Modeling Language*).

A **UML** é uma linguagem gráfica de modelagem padronizada em 1997 pelos chamados *UML Partners* e mantida pelo *Object Management Group* (OMG). Ela oferece um conjunto de diagramas (de classes, de sequência, de atividade, de casos de uso, entre outros) para descrever a estrutura e o comportamento de sistemas orientados a objetos. Os autores deste livro adotam a UML em todos os exemplos de modelagem OO a partir do Capítulo 16.

(e) Modela componentes de software em termos de objetos do mundo real

Resposta: Projeto Orientado a Objeto (OOD, *Object-Oriented Design*).

O **OOD** é a técnica de análise e de projeto na qual identificamos, no *domínio do problema*, as entidades relevantes (“Cliente”, “Conta”, “Pedido”, “Veículo”), descrevemos seus *atributos* (dados que cada uma possui) e suas *operações* (comportamentos que cada uma realiza), e mapeamos esse modelo conceitual para um conjunto de *classes* de uma linguagem OO. Quando esse trabalho é bem feito, o código resultante exibe uma correspondência natural com o problema de origem, o que aumenta a manutenibilidade e a reutilização.

Observação de Engenharia de Software

OOD complementa, mas não substitui, a *análise de requisitos*, que é a fase prévia em que se descobre o que o sistema deve fazer. A sequência típica é: análise de requisitos → OOAD (*Object-Oriented Analysis and Design*) → implementação em uma linguagem OO → teste → manutenção.

(f) Tipos definidos pelo programador em C++

Resposta: classes.

A **classe** é o conceito central de C++. Uma classe é, ao mesmo tempo, um *tipo* (no sentido em que `int` é um tipo) e um *molde* (a partir do qual se criam objetos individuais). Ela encapsula **dados-membro** (atributos) e **funções-membro** (métodos, operações) em uma única unidade. Esse é o veículo pelo qual C++ implementa o *encapsulamento* e o *ocultamento de informações* mencionados na Seção 15.10.

Observação de Engenharia de Software

A própria linguagem C++ é considerada *extensível* precisamente porque ela permite ao programador criar tipos definidos pelo usuário que “são tratados como cidadãos de primeira classe” — recebem operadores sobrecarregados, podem ser passados por valor ou por referência, podem ser usados em templates etc. Em C puro, não há esse nível de integração. Veremos no Capítulo 16 como criar nossa primeira classe.

2 Exercício 15.2 — Significado de `double &` num protótipo

Por que um protótipo de função contém uma declaração de parâmetro do tipo `double &`?

Resposta oficial: o tipo `double &` cria um **parâmetro de referência** para `double`, permitindo que a função modifique a variável original na função que a chamou.

Comentário didático

Em C puro, a única forma de uma função modificar uma variável do chamador é receber um **ponteiro**: o chamador passa o endereço (`&x`), a função desreferencia (`*p`). Funciona, mas a sintaxe é desajeitada:

```
/* C: estilo com ponteiro */
void dobra(int *p) { *p *= 2; }
int main() { int x = 5; dobra(&x); }    /* x agora vale 10 */
```

C++ oferece uma alternativa mais elegante: o **parâmetro de referência**. Declarado com `&` no tipo do parâmetro, ele cria um *alias* (segundo nome) para o argumento real do chamador:

```
// C++: estilo com referencia
void dobra(int &r) { r *= 2; }
int main() { int x = 5; dobra(x); }    // x agora vale 10
```

Compare lado a lado:

Aspecto	Ponteiro (C/C++)	Referência (C++)
Declaração	<code>int *p</code>	<code>int &r</code>
Chamada	<code>f(&x)</code>	<code>f(x)</code>
Uso dentro da função	<code>*p = 10</code>	<code>r = 10</code>
Pode ser NULL?	Sim	Não
Pode ser reatribuído?	Sim	Não (<i>aliasing</i> fixo)

Boa Prática de Programação

Use **referências** quando o argumento sempre existir e não precisar ser reatribuído durante a função — isto é, no caso típico de “modificar este objeto que recebi”. Use **ponteiros** quando o argumento puder ser opcional (`nullptr` válido), ou quando você precisar percorrer arrays, ou quando o objeto apontado puder mudar durante a função.

Erro Comum de Programação

Referências *pendentes* (*dangling references*) são um perigo sutil: ocorrem quando se retorna uma referência para uma variável local — a referência continua sintaticamente válida mas aponta para memória que foi liberada. O resultado é comportamento indefinido.

```
int &perigo() {
    int x = 42;
    return x;      // ERRO: x sera destruido ao sair desta funcao
}
```

Bons compiladores avisam (com a flag `-Wreturn-local-addr`). Trate esse aviso como erro fatal.

3 Exercício 15.3 — Verdadeiro ou Falso

(V/F) Todos os argumentos de chamadas de função em C++ são passados por valor.

Resposta: Falso.

Comentário didático

Em C, a sentença seria *verdadeira*: a linguagem oferece somente passagem por valor. Para simular passagem por referência, o programador C deve passar um ponteiro — mas o *ponteiro em si é passado por valor*.

C++, ao contrário, adicionou a **passagem por referência** como um mecanismo de primeira classe. Você pode escolher, parâmetro a parâmetro, qual modo usar:

- **Por valor:** `void f(int x)` — a função recebe uma cópia.
- **Por referência:** `void f(int &x)` — a função opera sobre a variável original.
- **Por referência constante:** `void f(const int &x)` — a função lê o original sem cópia, mas não pode modificá-lo.

Dica de Desempenho

A passagem por valor de objetos *grandes* (estruturas com muitos campos, strings, vetores) tem custo proporcional ao tamanho. Para objetos grandes que a função *não* irá modificar,

a passagem por `const` referência (`const T &`) é o idioma C++ canônico: evita a cópia e garante imutabilidade.

Observação de Engenharia de Software

Esta diferença em relação a C é tão significativa que justifica o subtítulo deste capítulo — “*um C melhor*”: mesmo antes de usarmos objetos, já temos em C++ um arsenal de melhorias procedurais (referências, sobrecarga, argumentos default, templates) que tornam o código mais legível e seguro.

4 Exercício 15.4 — Volume da Esfera com Função Inline

Escreva um programa completo que peça o raio de uma esfera e calcule e imprima seu volume. Use uma função *inline* `sphereVolume` que retorna $(4,0/3,0) \cdot 3,14159 \cdot \text{pow}(\text{radius}, 3)$.

Análise do problema

A fórmula do volume da esfera é $V = \frac{4}{3}\pi r^3$. A presença do qualificador `inline` indica que queremos sugerir ao compilador que *expand*a o código da função no local da chamada, em vez de gerar uma chamada de função convencional — evitando o custo do salto, salvamento de registradores e retorno. Para funções muito pequenas (como uma fórmula de uma linha), `inline` pode ser vantajoso. Para funções grandes, é contraproducente, pois infla o código executável.

Código em C++

```
1 // Exercício 15.4 - Volume de uma esfera com funcao inline
2 #include <iostream>
3 #include <cmath>
4
5 using namespace std;
6
7 const double PI = 3.14159;
8
9 // Funcao inline que calcula o volume de uma esfera
10 inline double sphereVolume(const double radius) {
11     return (4.0 / 3.0) * PI * pow(radius, 3);
12 }
13
14 int main() {
15     double radiusValue;
16
17     cout << "Digite o raio da esfera: ";
18     cin >> radiusValue;
19
20     cout << "Volume da esfera com raio " << radiusValue
21          << " e " << sphereVolume(radiusValue) << endl;
22
23     return 0;
24 }
```

Execução

Saída da execução

```
$ ./ex15_04
Digite o raio da esfera: 2
Volume da esfera com raio 2 e 33.5103
```

Discussão

Conferindo: $V = \frac{4}{3} \cdot 3,14159 \cdot 2^3 = \frac{4}{3} \cdot 3,14159 \cdot 8 \approx 33,5103$. Bate exatamente.

Aspectos a observar:

- `const double PI` declara uma constante global. Em C++, `const` é preferível ao `#define` do pré-processador, por respeitar tipos e escopos.
- `cin » radiusValue` lê da entrada padrão usando o **operador de extração de *stream*** ». É o análogo C++ do `scanf` de C, porém segura quanto a tipos: o compilador escolhe a versão apropriada de » para `double`.
- `cout « ...« endl` usa o **operador de inserção** « *encadeado* (em cascata). O `endl` é um *manipulador de stream* que insere uma quebra de linha e descarrega o *buffer*.

Observação de Engenharia de Software

A palavra-chave `inline` é uma *sugestão* ao compilador, não uma ordem. Compiladores modernos já inserem em linha automaticamente funções pequenas, mesmo sem o qualificador. O verdadeiro papel atual de `inline` é permitir que uma função seja definida em um *arquivo de cabeçalho* sem violar a regra de definição única (*One Definition Rule*). Os autores apresentam `inline` aqui por razões históricas: a intenção original em C++ era exatamente a otimização manual.

Boa Prática de Programação

Para constantes matemáticas como π , prefira `<cmath>` junto com o macro `M_PI` (POSIX) ou, em C++20, `std::numbers::pi`. Em qualquer caso, declarar `const double PI = 3.14159;` é perfeitamente adequado para fins didáticos e para programas pequenos.

Síntese conceitual

Os quatro exercícios de autorrevisão do Capítulo 15 cumprem juntos um papel introdutório panorâmico: **15.1** apresenta o vocabulário básico (sobrecarga, `::`, templates, UML, OOD, classes); **15.2** introduz referências; **15.3** contrasta C e C++ na questão da passagem de argumentos; e **15.4** provê o primeiro programa-completo em C++ deste livro — mais especificamente, um programa C++ que ainda não usa classes, mas já usa `cin`, `cout`, `inline`, e `const`.

É uma transição calculada: o leitor que dominou os Capítulos 1 a 14 sente-se imediatamente em casa, mas começa a perceber as melhorias sintáticas. O verdadeiro salto conceitual — a definição da primeira classe — virá no Capítulo 16. Os exercícios principais (15.5 a 15.10) consolidam, agora em código, os mesmos temas.